

Objects

An `object` groups data with the functions that work on it. Fields can have defaults, so making one is short.

```
use std.io

object Counter {
  name: Text = "counter"
  value: Int = 0

  fn bump(self) {
    self.value = self.value + 1
  }

  fn report(self) -> Text => "{self.name} is at {self.value}"
}

fn main() {
  var hits = Counter { name: "page views" }
  hits.bump()
  hits.bump()
  io.say(hits.report())
}
```

Every method that touches the object takes `self` as its first parameter. Writing it out keeps it obvious which functions belong to the instance.

`data` for plain values

When a type is only a bag of fields, use `data`. It has no methods and compares by value.

```
use std.io

data Point {
  x: Num
  y: Num
}

fn main() {
  let a = Point { x: 1.0, y: 2.0 }
  let b = Point { x: 1.0, y: 2.0 }
  io.say("{a.x}, {a.y}")
  io.say("same: {a == b}")
}
```

Composition first

Rather than building deep hierarchies, put one object inside another.

```

use std.io

object Engine {
  power: Int = 90
  fn describe(self) -> Text => "{self.power} hp"
}

object Car {
  make: Text = "saloon"
  engine: Engine = Engine { }
  fn describe(self) -> Text => "{self.make}, {self.engine.describe()}"
}

fn main() {
  let car = Car { make: "estate", engine: Engine { power: 140 } }
  io.say(car.describe())
}

```

Extending

`extends` is there when you genuinely want a specialised version. The child gets the parent's fields and can replace its methods.

```

use std.io

object Shape {
  name: Text = "shape"
  fn area(self) -> Num => 0.0
  fn describe(self) -> Text => "{self.name} with area {self.area()}"
}

object Square extends Shape {
  side: Num = 1.0
  fn area(self) -> Num => self.side * self.side
}

fn main() {
  let s = Square { name: "square", side: 3.0 }
  io.say(s.describe())
}

```

Exercise

Model a bank account with a balance, `deposit`, `withdraw` and a method that refuses to go below zero.